

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
-----o0o-----



SOICT

Báo Cáo Bài tập lớn
Lưu trữ và xử lý dữ liệu lớn

Đề tài
Hệ thống thu thập, phân tích xu hướng và
cảm xúc thị trường tiền mã hóa thông qua
nền tảng Twitter/X

Nhóm 16

Sinh viên thực hiện: 20235256 – Trần Dương An
20235281 – Cao Chí công
20235331 – Nguyễn Hữu Hiệu
20235427 – Vũ Quang Thắng
20235452 – Hoàng Minh Tuấn

Giảng viên hướng dẫn: TS. Trần Việt Trung

Hà Nội 6/2026

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI.....	5
1.1 Đặt vấn đề và tính cấp thiết.....	5
1.2 Mục tiêu nghiên cứu.....	5
1.3 Phạm vi và đối tượng nghiên cứu.....	6
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ.....	7
2.1 Kiến trúc xử lý dữ liệu lớn	7
2.1.1 Kiến trúc Lambda.....	7
2.1.2 So sánh với kiến trúc Kappa.....	7
2.2 Apache Kafka	7
2.3 Apache Spark	7
2.4 HDFS (Hadoop Distributed File System)	8
2.5 MongoDB.....	8
2.6 Docker và Kubernetes	8
2.7 FastAPI và React.....	8
2.8 Phân tích cảm xúc và phát hiện spam/bot	8
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	10
3.1 Yêu cầu hệ thống.....	10
3.1.1 Yêu cầu chức năng	10
3.1.2 Yêu cầu phi chức năng	10
3.2 Kiến trúc tổng thể.....	10
3.3 Luồng dữ liệu end-to-end	11
3.4 Thiết kế tổng quan các tầng.....	11
3.4.1 Tầng thu thập (Ingestion)	11
3.4.2 Tầng xử lý dòng (Speed Layer).....	11
3.4.3 Tầng xử lý lô (Batch Layer)	12
3.4.4 Tầng lưu trữ và phục vụ	12
3.4.5 Triển khai	12
3.5 Lựa chọn công nghệ và lý do	12
CHƯƠNG 4. TẦNG THU THẬP DỮ LIỆU (DATA INGESTION)	13
4.1 Tổng quan và vai trò.....	13
4.2 Công nghệ sử dụng.....	13
4.3 Thiết kế tầng thu thập.....	13
4.3.1 Hai luồng dữ liệu và topic Kafka	13
4.3.2 Cấu trúc bản tin (Message Schema).....	14
4.4 Cài đặt chi tiết	14
4.4.1 Gọi API chịu lỗi với nhiều khóa (Key Fallback).....	14

4.4.2 Kafka Producer với SSL.....	14
4.4.3 Trình thu thập thị trường (Market Crawler)	15
4.4.4 Cào dữ liệu lịch sử (Historical Backfill)	15
4.4.5 Trạm thu Kafka-to-HDFS (Streaming Sink)	15
4.5 Khó khăn gặp phải và cách khắc phục	16
4.6 Đánh giá	16
CHƯƠNG 5. TẦNG XỬ LÝ DÒNG (SPEED LAYER).....	17
5.1 Tổng quan.....	17
5.3 Chuẩn hóa và làm sạch dữ liệu.....	17
5.4 Kiểm soát chất lượng (Bad Records)	18
5.5 Tổng hợp chỉ số xu hướng theo cửa sổ thời gian	18
5.6 Phát hiện spike (đột biến).....	18
5.7 Ghi kết quả và đảm bảo nhất quán	19
5.8 Khó khăn gặp phải và cách khắc phục	19
CHƯƠNG 6. TẦNG XỬ LÝ LÔ (BATCH LAYER)	20
6.1 Tổng quan.....	20
6.2 Đọc dữ liệu và xử lý gia tăng (Incremental).....	20
6.3 Tách coin từ tweet đa coin	20
6.4 Lọc spam/bot bằng UDF	20
6.5 Phân tích cảm xúc và tổng hợp chỉ số	20
6.6 Phát hiện spike và ghi kết quả.....	21
6.7 Tối ưu và lập lịch.....	21
6.8 Khó khăn gặp phải và cách khắc phục	21
CHƯƠNG 7. TẦNG LƯU TRỮ VÀ PHỤC VỤ	22
7.1 Tổng quan.....	22
7.2 Kho dữ liệu thô — HDFS	22
7.3 Kho phục vụ — MongoDB	22
7.4 Dashboard Backend (FastAPI).....	23
7.5 Dashboard Frontend (React + Nginx)	24
7.6 Khó khăn gặp phải và cách khắc phục	24
CHƯƠNG 8. TRIỂN KHAI HỆ THỐNG TRÊN KUBERNETES	26
8.1 Tổng quan và lý do lựa chọn.....	26
8.3 Đóng gói Docker	27
8.4 Quy trình build và nạp image.....	27
8.5 Quản lý cấu hình bí mật (Secret).....	27
8.6 Một số manifest tiêu biểu.....	27
8.7 Lập lịch và truy cập.....	28

CHƯƠNG 9. KẾT QUẢ THỰC HIỆN VÀ ĐÁNH GIÁ	29
9.1 Kết quả triển khai	29
9.2 Số liệu thực nghiệm.....	29
9.3 Đánh giá hệ thống	30
9.4 Kỹ thuật Spark đã vận dụng	30
9.5 Khó khăn lớn gặp phải và cách khắc phục	31
CHƯƠNG 10. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	32
10.1 Kết luận	32
10.2 Bài học kinh nghiệm	32
10.3 Hạn chế.....	32

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI

1.1 Đặt vấn đề và tính cấp thiết

Thị trường tiền mã hóa (cryptocurrency) là một trong những thị trường tài chính biến động mạnh nhất, trong đó **tâm lý đám đông trên mạng xã hội** đóng vai trò dẫn dắt giá đặc biệt rõ rệt. Khác với thị trường chứng khoán truyền thống nơi giá phản ánh kết quả kinh doanh, giá của nhiều đồng coin chịu ảnh hưởng nặng từ các 'làn sóng' thảo luận trên X (Twitter), Reddit hay Telegram. Một dòng trạng thái của người có ảnh hưởng (KOL) hoặc một đợt thảo luận đột biến (spike) về một đồng coin có thể tạo biến động giá lớn chỉ trong vài giờ.

Vì vậy, **theo dõi và phân tích xu hướng thảo luận theo thời gian thực** có giá trị thiết thực: giúp nhà đầu tư nhận biết sớm các đợt quan tâm bất thường, và giúp nhà nghiên cứu định lượng được mối quan hệ giữa tâm lý mạng xã hội và thị trường. Tuy nhiên, bài toán này mang đầy đủ các đặc trưng '5V' của **dữ liệu lớn (Big Data)**:

- **Volume (Khối lượng)**: Lượng dữ liệu text thô kèm siêu dữ liệu (Metadata tương tác, thông tin cấu trúc User, followers, mốc thời gian) từ Twitter API đổ về đạt quy mô hàng triệu bản ghi mỗi ngày, tích lũy nhanh chóng chạm ngưỡng hàng trăm GB đến TB dữ liệu phi cấu trúc cần lưu trữ lâu dài.
- **Velocity (Tốc độ)**: Tốc độ thảo luận và biến động thị trường diễn ra từng giây. Hệ thống bắt buộc phải xử lý luồng (Streaming) để đưa ra cảnh báo đột biến (Spikes) dưới dạng near real-time, đồng thời vận hành song song lớp xử lý lô (Batch) theo chu kỳ giờ/ngày để cập nhật baseline lịch sử.
- **Variety (Đa dạng)**: Văn bản thô từ Twitter chứa cấu trúc ngôn ngữ tự nhiên phi chuẩn mực, bao gồm từ lóng đặc thù ngành (HODL, REKT, Paper Hands, Mooning), các mã ký hiệu tài sản (*Cashtags* như \$BTC, \$ETH) xen lẫn mật độ biểu tượng cảm xúc (Emojis) dày đặc phản ánh trực tiếp tâm lý người dùng.
- **Veracity (Độ tin cậy)**: Tỷ lệ nhiễu cực cao. Ước tính có tới hơn 35% lượng bài đăng đến từ mạng lưới tài khoản ảo tự động (Botnets/Spam) được vận hành nhằm mục đích thao túng tâm lý (Shilling/FUD) hoặc phát tán liên kết lừa đảo. Do đó, việc thiết kế các thuật toán lọc nhiễu Heuristic là bắt buộc để đảm bảo độ sạch của dữ liệu.
- **Value (Giá trị)**: Hệ thống chuyển hóa luồng thông tin hỗn loạn thành các chỉ số định lượng có tính ứng dụng cao, giúp Dashboard hiển thị trực quan các vùng tâm lý cực đoan (Extreme Fear/Extreme Greed) và phát hiện sớm các dòng tiền thảo luận trước khi biến động phản ánh lên đồ thị giá.

Một công cụ xử lý đơn lẻ khó đáp ứng đồng thời hai yêu cầu mâu thuẫn: **độ trễ thấp** (low latency) để phản hồi tức thời và **thông lượng cao** (high throughput) để phân tích sâu trên toàn bộ lịch sử. Do đó, cần một **hệ thống pipeline dữ liệu lớn hoàn chỉnh, end-to-end** với kiến trúc phù hợp.

1.2 Mục tiêu nghiên cứu

Đề tài hướng đến xây dựng một hệ thống phân tích xu hướng tiền mã hóa hoàn chỉnh theo **kiến trúc Lambda**, giải quyết các mục tiêu cốt lõi:

- Xây dựng **tầng thu thập** dữ liệu mạng xã hội về crypto theo thời gian gần thực và đẩy vào hàng đợi Kafka.

- Triển khai **tầng xử lý dòng (Speed Layer)** bằng Spark Structured Streaming để tính chỉ số xu hướng tức thời và phát hiện spike.
- Triển khai **tầng xử lý lô (Batch Layer)** bằng Spark để phân tích cảm xúc, lọc spam/bot và tổng hợp chỉ số trên dữ liệu lịch sử.
- Xây dựng **tầng lưu trữ và phục vụ**: HDFS làm kho thô, MongoDB làm kho phục vụ, và Dashboard trực quan hóa kết quả.
- **Triển khai toàn bộ hệ thống trên Kubernetes**, thể hiện khả năng vận hành một hệ thống nhiều thành phần có tính sẵn sàng và tự phục hồi.

1.3 Phạm vi và đối tượng nghiên cứu

- **Đối tượng nghiên cứu**: các công nghệ dữ liệu lớn (Apache Kafka, Apache Spark, HDFS), cơ sở dữ liệu NoSQL (MongoDB), công nghệ ảo hóa và điều phối container (Docker, Kubernetes), và kỹ thuật phân tích văn bản (sentiment analysis, phát hiện spam/bot).
- **Phạm vi dữ liệu**: dữ liệu công khai từ nền tảng X (Twitter) khai thác qua RapidAPI, giới hạn ở danh mục coin tiêu biểu (BTC, ETH, SOL, XRP, ADA, BNB, DOGE, AVAX).
- **Giới hạn**: phân tích cảm xúc dựa trên từ điển (lexicon-based) thay vì học sâu; hệ thống tập trung phát hiện xu hướng và cảm xúc, chưa dự báo giá; cụm Kubernetes triển khai trên hạ tầng cục bộ (Kind) phục vụ thử nghiệm.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1 Kiến trúc xử lý dữ liệu lớn

2.1.1 Kiến trúc Lambda

Kiến trúc Lambda là mô hình xử lý dữ liệu lớn kết hợp xử lý theo lô và xử lý dòng nhằm cân bằng giữa độ trễ thấp và độ chính xác/toàn diện. Kiến trúc gồm ba tầng:

- **Batch Layer (tầng lô):** xử lý toàn bộ dữ liệu lịch sử, cho kết quả chính xác và đầy đủ nhưng có độ trễ vì chạy định kỳ.
- **Speed Layer (tầng tốc độ):** xử lý dòng dữ liệu mới đến theo thời gian thực, cho kết quả tức thời (có thể gần đúng) để bù đắp độ trễ của tầng lô.
- **Serving Layer (tầng phục vụ):** hợp nhất kết quả của hai tầng và phục vụ truy vấn từ ứng dụng người dùng.

2.1.2 So sánh với kiến trúc Kappa

Một lựa chọn thay thế là **kiến trúc Kappa**, vốn chỉ dùng một tầng xử lý dòng duy nhất và xử lý lại (reprocessing) dữ liệu khi cần bằng cách phát lại luồng. Kappa đơn giản hơn về vận hành nhưng khó tối ưu cho các phân tích nặng trên toàn bộ lịch sử. Đề tài chọn **Lambda** vì bài toán cần cả phản hồi nhanh (phát hiện spike thời gian thực) lẫn phân tích sâu định kỳ (xu hướng cảm xúc theo coin), và việc tách hai tầng giúp mỗi tầng được tối ưu riêng.

2.2 Apache Kafka

Apache Kafka là nền tảng truyền tin nhắn phân tán theo mô hình **Publish/Subscribe**, được thiết kế cho thông lượng cao và độ bền dữ liệu. Các khái niệm chính:

- **Producer / Consumer:** bên gửi và bên nhận tin nhắn.
- **Broker:** máy chủ Kafka lưu trữ và phân phối tin nhắn.
- **Topic & Partition:** topic là kênh phân loại tin nhắn, được chia thành nhiều partition để xử lý song song.
- **Offset:** vị trí đọc của consumer trong partition; cho phép consumer kiểm soát tiến độ và đọc lại khi cần.
- **Consumer Group:** nhóm consumer cùng tiêu thụ một topic; mỗi partition chỉ giao cho một consumer trong nhóm, bảo đảm không xử lý trùng.

Kafka cho phép **tách rời (decoupling)** producer và consumer, đệm dữ liệu khi tải tăng đột biến, và bảo đảm độ bền nhờ lưu trữ bền vững cùng cơ chế offset/commit. Trong project, Kafka là 'xương sống' trung chuyển dữ liệu giữa tầng thu thập và các tầng xử lý; kết nối được mã hóa bằng SSL/TLS.

2.3 Apache Spark

Apache Spark là công cụ xử lý dữ liệu lớn trong bộ nhớ (in-memory), nhanh hơn nhiều so với MapReduce nhờ giảm số lần ghi đĩa trung gian. Mô hình tính toán của Spark dựa trên các **phép biến đổi (transformation)** có tính 'lazy' (chỉ định nghĩa kế hoạch) và các **hành động (action)** kích hoạt thực thi thực sự.

- **Spark SQL & DataFrame:** mô hình dữ liệu dạng bảng với API phong phú (select, filter, groupBy, agg, join, window...), hỗ trợ hàm tổng hợp nâng cao và **UDF (User Defined Function)** để cài đặt logic nghiệp vụ tùy biến.
- **Spark Structured Streaming:** xử lý dòng dữ liệu liên tục theo mô hình **micro-batch**, hỗ trợ **cửa sổ thời gian (windowing)**, **watermark** để xử lý dữ liệu đến trễ, **quản lý trạng thái (state)** và **checkpoint** để phục hồi, cùng các chế độ xuất (output mode) append/update/complete.

Project sử dụng **PySpark** (API Python của Spark) cho cả tầng lô và tầng dòng, tận dụng được hệ sinh thái Python phong phú cho xử lý văn bản.

2.4 HDFS (Hadoop Distributed File System)

HDFS là hệ thống tệp phân tán, chia tệp thành các khối (block) và lưu nhân bản trên nhiều node để chịu lỗi. Kiến trúc gồm **NameNode** (quản lý metadata, cây thư mục) và **DataNode** (lưu khối dữ liệu thực). Trong project, HDFS đóng vai trò **kho dữ liệu thô (raw zone)** lưu tweet dưới dạng JSON Lines, có **phân vùng theo coin/ngày/giờ** giúp tầng lô chỉ đọc đúng phần dữ liệu cần (partition pruning). Hệ thống truy cập HDFS qua giao thức **WebHDFS** (REST trên HTTP) để không phụ thuộc vào binary Hadoop cục bộ.

2.5 MongoDB

MongoDB là cơ sở dữ liệu **NoSQL** hướng tài liệu (document-oriented), lưu dữ liệu dạng JSON/BSON linh hoạt, không yêu cầu schema cố định — phù hợp với dữ liệu chỉ số có cấu trúc thay đổi và truy vấn tổng hợp (aggregation) nhanh. Trong project, MongoDB là **tầng phục vụ**, lưu các collection kết quả như `speed_trend_metrics`, `batch_sentiment_metrics`, `alerts`, `batch_trend_spikes`... để Dashboard truy vấn.

2.6 Docker và Kubernetes

Docker là công nghệ ảo hóa mức hệ điều hành (OS-level virtualization), đóng gói ứng dụng cùng toàn bộ phụ thuộc vào **container** cách ly, bảo đảm tính nhất quán giữa môi trường phát triển và triển khai. Khác với máy ảo, container chia sẻ nhân hệ điều hành nên nhẹ và khởi động nhanh.

Kubernetes là nền tảng **điều phối container (container orchestration)**, tự động hóa triển khai, mở rộng, tự phục hồi (self-healing) và quản lý vòng đời container thông qua các đối tượng khai báo (Deployment, StatefulSet, CronJob, Job, Service, Secret...). Project dùng Docker để đóng gói từng tầng và Kubernetes để vận hành toàn hệ thống một cách tự động và bền bỉ.

2.7 FastAPI và React

Tầng trực quan hóa gồm **FastAPI** — framework web Python bất đồng bộ, hiệu năng cao, tự sinh tài liệu API — làm backend cung cấp REST API truy vấn MongoDB; và **React (Vite)** làm frontend hiển thị dashboard trực quan (thẻ KPI, biểu đồ xu hướng, bản đồ nhiệt cảm xúc, danh sách cảnh báo) với khả năng tự làm mới định kỳ.

2.8 Phân tích cảm xúc và phát hiện spam/bot

Phân tích cảm xúc (sentiment analysis) sử dụng cách tiếp cận **dựa trên từ điển (lexicon-based)** kiểu VADER: mỗi tweet được chấm điểm cảm xúc tổng hợp (compound) trong khoảng $[-1, 1]$, rồi phân loại bullish/neutral/bearish theo ngưỡng, từ đó suy ra chỉ số tâm lý **Fear & Greed**. Ưu điểm của cách tiếp cận này là nhẹ, không cần huấn luyện, phù hợp xử lý khối lượng

lớn theo thời gian thực; hạn chế là độ chính xác phụ thuộc từ điển và khó nắm bắt ngữ cảnh/tiếng lóng.

Phát hiện spam/bot dựa trên hệ luật và heuristic có trọng số: phát hiện lạm dụng emoji, spam hashtag/mention, ngập URL, từ khóa quảng cáo/lừa đảo, mẫu tên tài khoản đáng ngờ và bất thường về tương tác. Hai kỹ thuật này được áp dụng ở tầng lô để **làm sạch dữ liệu** trước khi tổng hợp chỉ số, bảo đảm chất lượng phân tích.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Yêu cầu hệ thống

3.1.1 Yêu cầu chức năng

- Thu thập tweet về tiền mã hóa từ mạng xã hội theo chu kỳ và đẩy vào hàng đợi Kafka.
- Lưu trữ dữ liệu thô bền vững xuống HDFS có phân vùng.
- Tính toán chỉ số xu hướng thời gian thực (mention, trend score) và phát hiện spike ở tầng dòng.
- Phân tích cảm xúc, lọc spam/bot và tổng hợp chỉ số Fear & Greed theo lô.
- Lưu kết quả vào MongoDB và hiển thị trực quan trên Dashboard web.

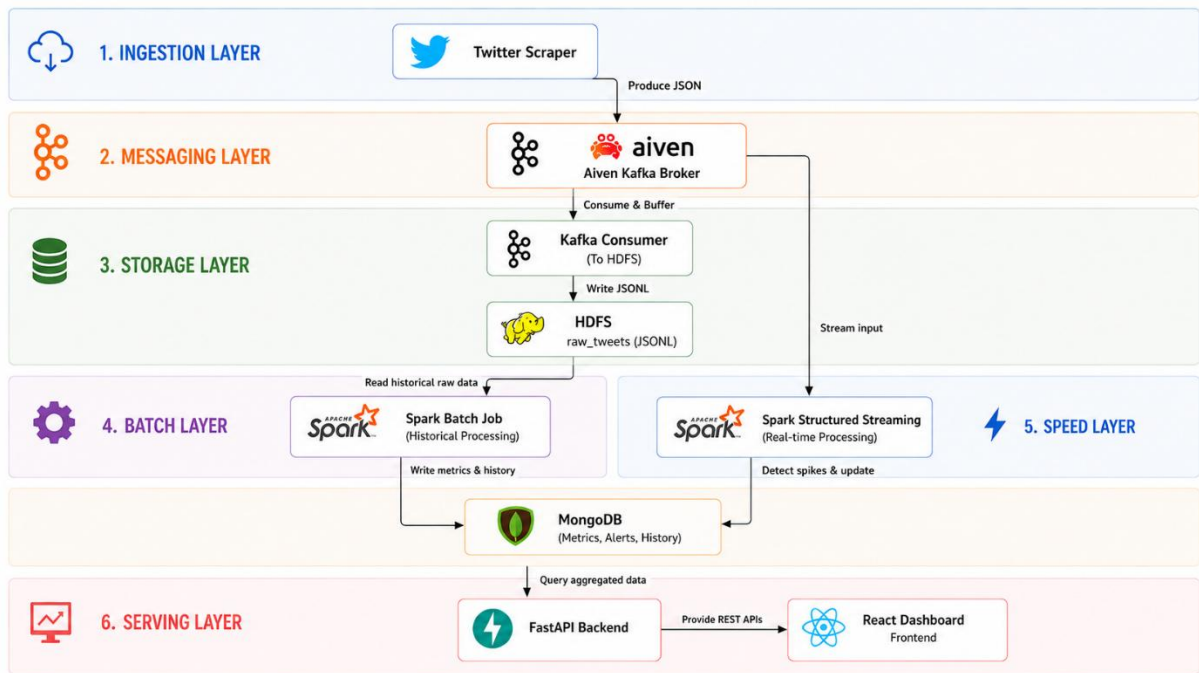
3.1.2 Yêu cầu phi chức năng

- **Khả năng mở rộng:** các tầng tách rời qua Kafka, có thể mở rộng độc lập.
- **Độ tin cậy:** chịu lỗi nguồn API, không mất dữ liệu khi tiến trình khởi động lại.
- **Tính sẵn sàng:** tự phục hồi khi container lỗi (nhờ Kubernetes).
- **Khả năng bảo trì:** cấu hình tách khỏi mã nguồn, đóng gói container nhất quán.

3.2 Kiến trúc tổng thể

Hệ thống được thiết kế theo **kiến trúc Lambda** với năm khối chức năng liên kết qua Kafka và các kho dữ liệu. Toàn bộ vận hành trên cụm Kubernetes.

Khối	Thành phần	Công nghệ
Thu thập (Ingestion)	Market crawler, Historical scraper, Kafka-to-HDFS sink	Python, kafka-python, RapidAPI
Hàng đợi (Messaging)	Topic raw-tweets-market / raw-tweets-whales	Apache Kafka (Aiven, SSL)
Lưu trữ thô (Raw zone)	Kho dữ liệu tweet phân vùng	HDFS (WebHDFS)
Xử lý dòng (Speed)	spark-stream-processor	Spark Structured Streaming
Xử lý lô (Batch)	spark-batch-processor	Spark Batch (PySpark)
Phục vụ (Serving)	MongoDB + Dashboard (FastAPI/React)	MongoDB, FastAPI, React
Vận hành	Toàn bộ container	Docker, Kubernetes (Kind)



3.3 Luồng dữ liệu end-to-end

Dữ liệu chảy qua hệ thống theo trình tự khép kín:

1. **Thu thập:** crawler gọi RapidAPI lấy tweet, chuẩn hóa thành JSON và **gửi lên Kafka** (topic raw-tweets-market).
2. **Trung chuyển:** Kafka đệm và phân phối tin nhắn cho các bên đăng ký.
3. **Lưu trữ thô:** tiến trình Kafka-to-HDFS gom lô và ghi tweet xuống **HDFS** theo phân vùng coin/ngày/giờ.
4. **Xử lý dòng:** Spark Structured Streaming đọc trực tiếp từ Kafka, tính chỉ số xu hướng theo cửa sổ trượt và phát hiện spike, ghi vào **MongoDB** (speed_trend_metrics).
5. **Xử lý lô:** định kỳ, Spark Batch đọc dữ liệu thô từ HDFS, lọc spam/bot, phân tích cảm xúc, tổng hợp chỉ số và ghi vào **MongoDB** (batch_sentiment_metrics, batch_trend_spikes, alerts...).
6. **Phục vụ:** Dashboard (FastAPI + React) truy vấn MongoDB và hiển thị KPI, biểu đồ, cảnh báo theo thời gian thực.

3.4 Thiết kế tổng quan các tầng

3.4.1 Tầng thu thập (Ingestion)

Khai thác hai luồng (market, whale) qua RapidAPI với cơ chế xoay vòng nhiều khóa API; chuẩn hóa schema tweet; gửi Kafka qua SSL; và một trạm sink ghi dữ liệu thô xuống HDFS theo lô với cơ chế at-least-once. (Chi tiết tại Chương 4.)

3.4.2 Tầng xử lý dòng (Speed Layer)

Spark Structured Streaming đọc Kafka, phân tích cú pháp JSON, làm sạch và chuẩn hóa, bóc tách cashtag, rồi tổng hợp theo **cửa sổ trượt** kèm watermark; tính trend_score và làm giàu thông tin spike (z-score, growth-rate) trước khi ghi MongoDB. (Chi tiết tại Chương 5.)

3.4.3 Tầng xử lý lô (Batch Layer)

Spark Batch đọc dữ liệu thô từ HDFS theo phân vùng (xử lý gia tăng - incremental), tách coin từ tweet đa coin, lọc spam/bot bằng UDF, chấm điểm cảm xúc, tổng hợp chỉ số theo (coin, cửa sổ thời gian) và phân tách nhóm whale/retail, sau đó ghi MongoDB. (Chi tiết tại Chương 6.)

3.4.4 Tầng lưu trữ và phục vụ

HDFS lưu dữ liệu thô; MongoDB lưu các chỉ số kết quả; Dashboard gồm backend FastAPI cung cấp API và frontend React trực quan hóa. (Chi tiết tại Chương 7.)

3.4.5 Triển khai

Toàn bộ thành phần được đóng gói Docker và triển khai trên Kubernetes với các đối tượng Deployment, CronJob, Job, StatefulSet, Service và Secret. (Chi tiết tại Chương 8.)

3.5 Lựa chọn công nghệ và lý do

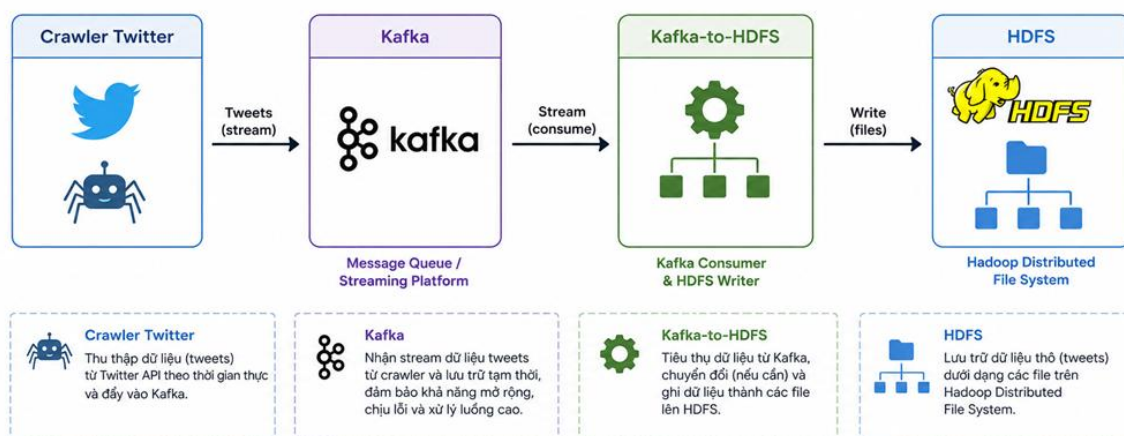
Nhu cầu	Công nghệ chọn	Lý do
Hàng đợi tin nhắn	Apache Kafka	Thông lượng cao, tách rời producer/consumer, bền dữ liệu
Xử lý lô & dòng	Apache Spark	Một nền tảng cho cả batch và streaming, API phong phú
Lưu trữ thô	HDFS	Phân tán, chịu lỗi, phân vùng tốt cho batch
Kho phục vụ	MongoDB	Linh hoạt schema, truy vấn nhanh cho dashboard
Trực quan hóa	FastAPI + React	Backend nhẹ, frontend hiện đại, tách biệt rõ ràng
Triển khai	Docker + Kubernetes	Nhất quán môi trường, tự phục hồi, quản lý tập trung

CHƯƠNG 4. TẦNG THU THẬP DỮ LIỆU (DATA INGESTION)

4.1 Tổng quan và vai trò

Tầng thu thập là **nguồn (source)** của toàn bộ pipeline, chịu trách nhiệm khai thác dữ liệu thô từ mạng xã hội, chuẩn hóa và bơm vào Kafka để các tầng phía sau tiêu thụ. Mục tiêu thiết kế: (1) thu thập near real-time; (2) chịu lỗi khi nguồn API bị giới hạn; (3) tách rời thu thập với xử lý qua Kafka; (4) lưu trữ bền vững xuống HDFS phục vụ tầng lô.

Sơ đồ tổng quan tầng thu thập



4.2 Công nghệ sử dụng

- **RapidAPI (Twitter API)**: khai thác tweet qua endpoint search.php dùng cú pháp Advanced Search (lọc theo cashtag \$BTC, tài khoản from:, ngôn ngữ lang:en, khoảng thời gian since_time/until_time).
- **Apache Kafka (Aiven)**: hàng đợi tin nhắn, kết nối qua SSL/TLS hai chiều với ba chứng chỉ ca.pem, service.cert, service.key.
- **Python 3.11** với thư viện kafka-python, requests, python-dotenv; cấu hình qua biến môi trường.

4.3 Thiết kế tầng thu thập

4.3.1 Hai luồng dữ liệu và topic Kafka

Luồng	Topic Kafka	Nội dung	Nhãn target_coin
Market Trend	raw-tweets-market	Tweet đề cập danh mục coin (\$BTC, \$ETH...)	MULTI_CRYPTO
Whale Signal	raw-tweets-whales	Tweet từ tài khoản có ảnh hưởng (KOL)	WHALE_SIGNAL

Danh mục coin (TRACKED_COINS): BTC, ETH, SOL, XRP, ADA, BNB, DOGE, AVAX — cấu hình động qua biến môi trường. Luồng Whale đã được lược bỏ khỏi lịch vận hành do đóng góp ít cho phân tích xu hướng coin.

4.3.2 Cấu trúc bản tin (Message Schema)

Mỗi tweet được chuẩn hóa thành một đối tượng JSON thống nhất. Ví dụ một bản tin thực tế gửi lên Kafka:

```
{
  "id": "2062197554347974872",
  "text": "$BTC to a Billion",
  "created_at": "Wed Jun 03 15:39:45 +0000 2026",
  "author": "TheFastTrader12",
  "target_coin": "MULTI_CRYPTO",
  "like_count": 0, "retweet_count": 0, "reply_count": 0,
  "quote_count": 0, "bookmark_count": 0,
  "followers_count": 42, "verified": true
}
```

Mã 4.1. Ví dụ bản tin tweet đã chuẩn hóa

4.4 Cài đặt chi tiết

4.4.1 Gọi API chịu lỗi với nhiều khóa (Key Fallback)

Các gói RapidAPI miễn phí bị giới hạn truy vấn (rate limit). Module `api_helper.py` hiện thực **xoay vòng khóa dự phòng**: danh sách khóa lấy từ biến `RAPIDAPI_KEYS`; khi một khóa trả mã 429 (Too Many Requests) hoặc 403 (Forbidden), hệ thống tự chuyển sang khóa kế tiếp. Đoạn code cốt lõi:

```
for idx, key in enumerate(valid_keys):
    headers = {"x-rapidapi-key": key, "x-rapidapi-host": RAPIDAPI_HOST}
    response = requests.get(url, headers=headers, params=querystring)
    if response.status_code == 200:
        return response.json()          # khóa còn tốt -> trả dữ liệu
    elif response.status_code in [429, 403]:
        time.sleep(1); continue         # hết hạn -> nạp khóa kế tiếp
return None                             # hết khóa -> dừng chu kỳ
```

Mã 4.2. Cơ chế xoay vòng khóa API (`api_helper.py`)

4.4.2 Kafka Producer với SSL

Module `kafka_connection.py` khởi tạo `KafkaProducer` kết nối Aiven qua SSL, với `acks='all'` để tối đa độ bền và `value_serializer` chuyển đối tượng Python sang JSON UTF-8:

```

producer = KafkaProducer(
    bootstrap_servers=f"{KAFKA_HOST}:{KAFKA_PORT}",
    security_protocol="SSL",
    ssl_cafile=os.path.join(CERTS_DIR, "ca.pem"),
    ssl_certfile=os.path.join(CERTS_DIR, "service.cert"),
    ssl_keyfile=os.path.join(CERTS_DIR, "service.key"),
    acks="all",
    value_serializer=lambda v: json.dumps(v).encode("utf-8"),
)

```

Mã 4.3. Khởi tạo Kafka Producer với SSL (*kafka_connection.py*)

Đường dẫn chứng chỉ lấy từ biến KAFKA_CERTS_DIR; trên Kubernetes biến này trỏ tới /etc/kafka/certs — nơi Secret kafka-ssl-secret được mount vào container.

4.4.3 Trình thu thập thị trường (Market Crawler)

Module twitter_client.py thu thập theo mô hình **pseudo-streaming**: mỗi lần chạy khai thác tweet trong cửa sổ một giờ gần nhất, dựng câu truy vấn Advanced Search, khử trùng lặp và gửi lên Kafka:

```

hien_tai = int(time.time()); thoi_gian_truoc = hien_tai - 3600
coin_query = " OR ".join([f"${c}" for c in coins])
cau_lenh = f'({coin_query}) -filter:replies lang:en "\
    f"since_time:{thoi_gian_truoc} until_time:{hien_tai}"
...
if tweet_id not in seen_tweets:      # khử trùng lặp (deque maxlen=2000)
    producer.send(KAFKA_TOPIC, key="CRYPTO", value=clean_tweet)
    seen_tweets.append(tweet_id)

```

Mã 4.4. Lỗi thu thập thị trường (*twitter_client.py*)

4.4.4 Cào dữ liệu lịch sử (Historical Backfill)

Module historical_scraper.py chạy **một lần** để nạp dữ liệu quá khứ, chia khoảng thời gian thành các 'lát' (market 1 giờ/lát, whale 24 giờ/lát) nhằm tiết kiệm khóa API; số ngày điều khiển bởi HISTORICAL_DAYS_TO_FETCH (180 ngày khi triển khai). Giữa các lát có thời gian nghỉ để tránh bị chặn.

4.4.5 Trạm thu Kafka-to-HDFS (Streaming Sink)

Module kafka_to_hdfs.py chạy **liên tục**, đăng ký KafkaConsumer lắng nghe các topic và ghi xuống HDFS theo lô. Cơ chế gom lô và bảo đảm không mất dữ liệu (at-least-once):

```

consumer = KafkaConsumer(*TOPICS, ...,
    auto_offset_reset="earliest",
    enable_auto_commit=False,          # tự quản lý commit để không mất dữ liệu
    group_id="hdfs-sink-group")
...
# ghi khi đủ 500 bản ghi HOẶC sau 60 giây
if len(buffer) >= BATCH_SIZE or time_elapsed >= BATCH_INTERVAL:
    for coin, tweets in grouped_tweets.items():
        hdfs_client.store_raw_tweets(coin, tweets)
    if success:
        consumer.commit()              # chỉ commit SAU KHI ghi HDFS thành công

```

Mã 4.5. Gom lô và commit thủ công (*kafka_to_hdfs.py*)

Trước khi ghi, tweet được nhóm theo `target_coin` và lưu HDFS theo cấu trúc phân vùng `coin=<COIN>/date=<YYYY-MM-DD>/hour=<HH>`, thuận lợi cho partition pruning ở tầng lô.

4.5 Khó khăn gặp phải và cách khắc phục

Khó khăn	Cách khắc phục
Khóa RapidAPI miễn phí nhanh hết hạn mức (429/403), gián đoạn thu thập	Xoay vòng nhiều khóa dự phòng (RAPIDAPI_KEYS), tự chuyển khóa khi gặp lỗi
Cấu trúc JSON trả về từ API không đồng nhất giữa các lần gọi	Dùng nhiều khóa thay thế (timeline/data/results/tweets) và <code>.get()</code> có giá trị mặc định
Tweet bị trùng lặp giữa các chu kỳ thu thập	Khử trùng lặp bằng hàng đợi deque(maxlen=2000) lưu id đã xử lý
Nguy cơ mất dữ liệu khi trạm sink khởi động lại	Tắt auto-commit, chỉ commit offset sau khi ghi HDFS thành công (at-least-once)
Quản lý chứng chỉ SSL trong container	Đưa cert vào Secret, mount vào /etc/kafka/certs, trả bằng KAFKA_CERTS_DIR

4.6 Đánh giá

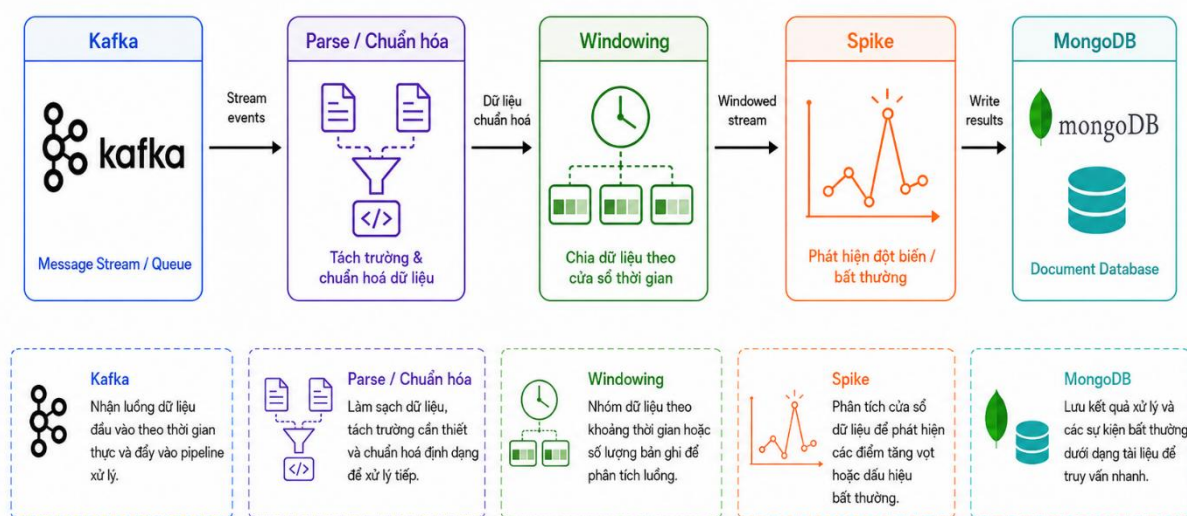
Tầng thu thập đáp ứng các mục tiêu: thu thập near real-time, chịu lỗi nhờ xoay vòng khóa và commit thủ công, tách rời qua Kafka, lưu trữ bền vững có phân vùng. Thực nghiệm cho thấy mỗi chu kỳ bơm thành công hàng chục tweet mới lên Kafka và được trạm sink ghi xuống HDFS ổn định.

CHƯƠNG 5. TẦNG XỬ LÝ DÒNG (SPEED LAYER)

5.1 Tổng quan

Tầng xử lý dòng tính toán **chỉ số xu hướng theo thời gian thực** và **phát hiện spike** ngay khi tweet mới chảy về. Tầng này hiện thực bằng **Spark Structured Streaming** (PySpark), đọc trực tiếp từ Kafka, xử lý theo micro-batch và ghi kết quả vào MongoDB để Dashboard hiển thị tức thời.

Sơ đồ pipeline tầng streaming processing



5.2 Đọc dữ liệu từ Kafka

Spark dùng `readStream` định dạng 'kafka', kết nối Aiven qua SSL với `truststore/keystore` dạng PEM. Tham số `startingOffsets='latest'` để chỉ xử lý tweet đến sau khi khởi động; value (kiểu nhị phân) được ép chuỗi rồi phân tích JSON theo schema:

```
parsed = (raw_df
    .selectExpr("topic", "CAST(value AS STRING) AS raw_json")
    .withColumn("json_data", F.from_json(F.col("raw_json"), schema))
    .select("raw_json", "json_data", "json_data.*"))
```

Mã 5.1. Phân tích JSON từ Kafka (spark_pipeline.py)

5.3 Chuẩn hóa và làm sạch dữ liệu

Do dữ liệu nguồn có nhiều biến thể tên trường, pipeline thực hiện chuỗi biến đổi nhiều giai đoạn: hợp nhất bí danh trường bằng `coalesce` (ví dụ `like_count` lấy từ `favorites/likes/favorite_count`), phân tích `event_time`, tính `engagement_score`, bóc `cashtag`, suy ra `author_weight` (`whale = 5.0`) và `influence_score` (`= author_weight × engagement_score`). Ví dụ tính `engagement_score`:

```
.withColumn("engagement_score",
  F.col("like_count") + F.col("retweet_count") + F.col("reply_count")
  + F.col("quote_count") + F.col("bookmark_count"))
```

Mã 5.2. Tính chỉ số tương tác tổng hợp

5.4 Kiểm soát chất lượng (Bad Records)

Mỗi bản ghi được gắn cờ `invalid_reason` nếu vi phạm (JSON hỏng, thiếu id, sai timestamp, thiếu nội dung, thiếu cashtag). Các bản ghi lỗi được tách riêng, áp **watermark 10 phút** và tổng hợp theo **cửa sổ tumbling 5 phút** theo loại lỗi, ghi vào collection `speed_bad_records` phục vụ giám sát chất lượng.

5.5 Tổng hợp chỉ số xu hướng theo cửa sổ thời gian

Các bản ghi hợp lệ được áp **watermark 10 phút** trên `event_time`, **khử trùng lặp** theo `tweet_id`, rồi explode danh sách cashtag thành từng dòng theo `symbol`:

```
cleaned = (normalized.filter(F.col("invalid_reason").isNull())
  .withWatermark("event_time", "10 minutes")
  .dropDuplicates(["tweet_id"])
  .withColumn("symbol", F.explode("cashtags")))
```

Mã 5.3. Watermark, khử trùng lặp và explode cashtag

Sau đó tổng hợp theo **cửa sổ trượt 5 phút, trượt mỗi 1 phút** kết hợp `symbol`, dùng `approx_count_distinct` (HyperLogLog) để đếm tác giả duy nhất — một tối ưu hiệu năng quan trọng — và tính `trend_score` theo trọng số:

```
trend_metrics = (cleaned.groupBy(
  F.window("event_time", "5 minutes", "1 minute"), F.col("symbol"))
  .agg(F.count("*").alias("mention_count"),
  F.approx_count_distinct("user_id").alias("unique_authors"),
  F.sum("engagement_score").alias("engagement_score"),
  F.round(F.sum("influence_score"), 2).alias("influence_score"))
  .withColumn("trend_score", F.round(
  F.col("mention_count") * 2.0 + F.col("unique_authors")
  + F.col("engagement_score") / 10.0 + F.col("influence_score") / 20.0, 2)))
```

Mã 5.4. Tổng hợp theo cửa sổ trượt và tính trend_score

5.6 Phát hiện spike (đột biến)

Trước khi ghi MongoDB, mỗi bản ghi được làm giàu thông tin spike (module `spike_detection`). Hệ thống truy vấn lịch sử cùng `symbol` để tính **đường cơ sở (baseline)** — trung bình và độ lệch chuẩn mention theo các cửa sổ cùng khung giờ — rồi tính `z-score` và `growth_rate`. Logic quyết định spike:

```
growth_rate = current_mentions / max(baseline_mentions, 1.0)
z_score = (current_mentions - baseline_mentions) / baseline_deviation
is_spike = has_volume and has_authors and not is_suppressed and (
  z_score >= 3.0 or growth_rate >= 3.0)
```

Mã 5.5. Điều kiện đánh dấu spike (spike_detection.py)

Điều kiện khối lượng tối thiểu (mention ≥ 10 , unique authors ≥ 3) cùng **cơ chế ức chế 30 phút** tránh báo spike lặp lại liên tục cho cùng một symbol.

5.7 Ghi kết quả và đảm bảo nhất quán

Kết quả ghi vào MongoDB qua **foreachBatch**, dùng **UpdateOne** với **upsert** theo khóa (symbol, window_start, window_end) — thao tác **idempotent** giúp đạt hiệu quả exactly-once: nếu micro-batch bị xử lý lại (khôi phục từ checkpoint), kết quả chỉ bị ghi đè chứ không nhân đôi:

```
UpdateOne({"symbol": s, "window_start": ws, "window_end": we},
          {"$set": enriched_document}, upsert=True)
```

Mã 5.6. Upsert idempotent vào MongoDB (stream_runtime.py)

Mỗi truy vấn streaming có checkpoint riêng để lưu offset Kafka và trạng thái cửa sổ. Hai truy vấn (chỉ số xu hướng và bad records) chạy song song với outputMode='update'. Ví dụ một document kết quả:

```
{ "symbol": "BTC", "mention_count": 12, "unique_authors": 9,
  "engagement_score": 3518, "trend_score": 38.6, "is_spike": false,
  "window_start": "2026-06-03T20:55:00", "window_end": "2026-06-03T21:00:00" }
```

Mã 5.7. Ví dụ document speed_trend_metrics

5.8 Khó khăn gặp phải và cách khắc phục

Khó khăn	Cách khắc phục
Dữ liệu đến trễ làm sai kết quả cửa sổ (tàng dòng)	Áp watermark 10 phút để Spark xử lý đúng dữ liệu trễ và dọn trạng thái cũ
Nguy cơ ghi trùng khi micro-batch xử lý lại	Ghi bằng upsert idempotent theo khóa cửa sổ (hiệu quả exactly-once)

CHƯƠNG 6. TẦNG XỬ LÝ LÔ (BATCH LAYER)

6.1 Tổng quan

Tầng xử lý lô phân tích **toàn diện trên dữ liệu lịch sử** trong HDFS, tập trung **phân tích cảm xúc, lọc spam/bot** và **tổng hợp chỉ số Fear & Greed** theo từng coin. Tầng này chạy định kỳ (CronJob mỗi giờ) bằng Spark Batch (PySpark), ghi kết quả vào MongoDB.

6.2 Đọc dữ liệu và xử lý gia tăng (Incremental)

Job đọc JSON Lines từ HDFS theo phân vùng coin=*/date=*/hour=*. Để tránh xử lý lại toàn bộ, hệ thống **xử lý gia tăng**: lấy mốc window_end lớn nhất đã xử lý trong MongoDB rồi chỉ sinh đường dẫn phân vùng chưa xử lý — chính là kỹ thuật **partition pruning**:

```
def build_hdfs_path(target_date=None, target_hour=None, base=HDFS_BASE_PATH):
    date_part = f'date={target_date}' if target_date else "date=*"
    hour_part = f'hour={target_hour}' if target_hour else "hour=*"
    return f'{base}/coin=*/{date_part}/{hour_part}/*.jsonl'
```

Mã 6.1. Sinh đường dẫn phân vùng (partition pruning)

6.3 Tách coin từ tweet đa coin

Một tweet có thể đề cập nhiều coin (nhãn MULTI_CRYPTO). Pipeline dùng regexp_extract_all bóc cashtag, lọc theo danh mục theo dõi, rồi **explode** thành nhiều dòng — mỗi dòng một coin; loại bỏ UNKNOWN.

6.4 Lọc spam/bot bằng UDF

Hệ thống xây dựng **UDF** bóc bộ lọc BotSpamFilter, trả về cấu trúc gồm is_spam, is_bot, spam_score và lý do. Bộ lọc chấm điểm theo nhiều nhóm dấu hiệu có trọng số (từ khóa spam/scam/bot, lạm dụng emoji, spam hashtag/mention, ngập URL, tên tài khoản đáng ngờ, bất thường tương tác):

```
def _filter_udf():
    def _fn(content, username, eng, weight):
        r = get_spam_filter().detect_spam(content or "", username=username,
            engagement_score=int(eng or 0), author_weight=float(weight or 1.0))
        return (r.is_spam, r.is_bot, float(r.total_score), r.reasons)
    return F.udf(_fn, filter_result_type)
```

Mã 6.2. UDF lọc spam/bot (batch_job.py)

Tweet vượt ngưỡng (0.4) bị tách khỏi dòng dữ liệu sạch. Đóng gói logic phức tạp trong UDF cho phép áp dụng song song trên toàn bộ dữ liệu Spark.

6.5 Phân tích cảm xúc và tổng hợp chỉ số

Mỗi tweet sạch được chấm điểm cảm xúc (UDF kiểu VADER) cho compound trong [-1,1], phân loại bullish (≥ 0.05)/bearish (≤ -0.05)/neutral; suy ra Fear & Greed = $(avg+1) \times 50$. Dữ liệu được tổng hợp theo (coin, cửa sổ giờ) với tổng hợp có điều kiện:

```
coin_metrics_df = df_analyzed.groupBy("coin", "time_window").agg(
    F.count("*").alias("mention_count"),
    F.round(F.avg("sentiment_score"), 4).alias("avg_sentiment"),
    F.sum(F.when(F.col("sentiment_score") >= 0.05, 1).otherwise(0))
    .alias("bullish_count"),
    F.sum("engagement_score").alias("total_engagement"))\
.withColumn("fear_greed_score", F.round((F.col("avg_sentiment")+1)*50, 2))
```

Mã 6.3. Tổng hợp chỉ số cảm xúc theo coin

Ngoài ra pipeline tổng hợp theo (coin, cửa sổ, author_type) để **phân tách Whale vs Retail** — so sánh tâm lý nhóm ảnh hưởng với nhà đầu tư nhỏ lẻ.

6.6 Phát hiện spike và ghi kết quả

Job so sánh mention_count mỗi coin với trung bình ngày hôm trước (từ MongoDB); vượt ngưỡng tuyệt đối và tỷ lệ tăng trưởng thì đánh dấu spike. Kết quả ghi vào MongoDB: batch_sentiment_metrics, alerts (spam), batch_trend_spikes, và batch_job_runs (nhật ký kiểm toán). Ví dụ một document:

```
{ "coin": "XRP", "mention_count": 3, "bullish_ratio": 0.3333,
  "bearish_ratio": 0.0, "fear_greed_score": 56.7, "total_engagement": 0,
  "window_start": "2026-06-03T16:00:00", "window_end": "2026-06-03T17:00:00" }
```

Mã 6.4. Ví dụ document batch_sentiment_metrics

6.7 Tối ưu và lập lịch

Spark được cấu hình spark.sql.shuffle.partitions=2 và bộ nhớ driver/executor phù hợp môi trường cục bộ để tránh lỗi hết bộ nhớ (OOM). Tăng lô triển khai dưới dạng CronJob chạy **mỗi 1 giờ**, đọc các phân vùng mới và cập nhật MongoDB.

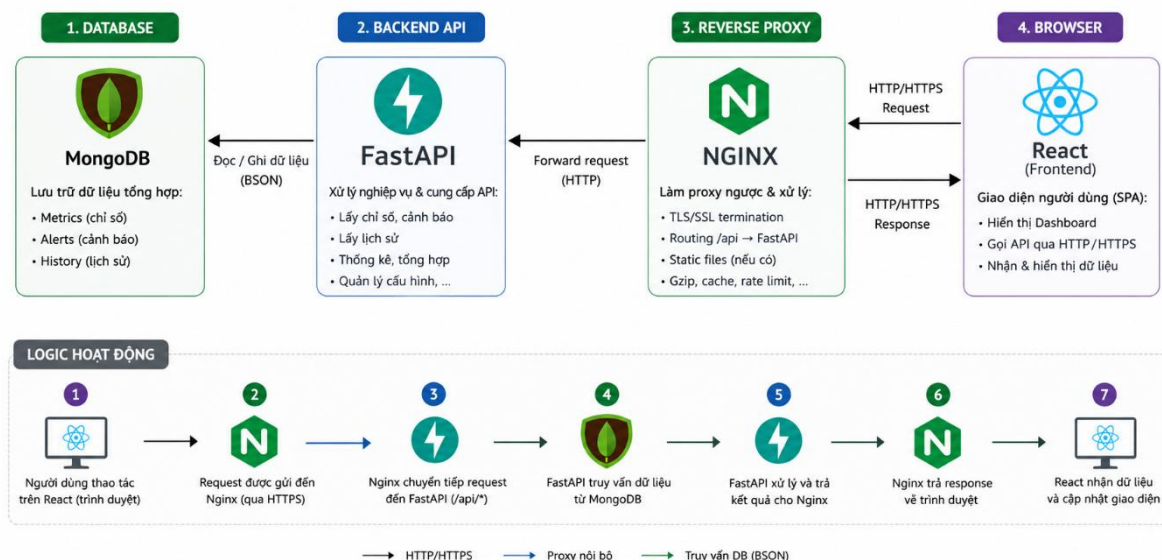
6.8 Khó khăn gặp phải và cách khắc phục

Khó khăn	Cách khắc phục
Tweet đề cập nhiều coin gây nhập nhằng khi tổng hợp	Bóc cashtag và explode thành nhiều dòng theo từng coin trước khi tổng hợp
Quét lại toàn bộ HDFS tốn tài nguyên	Xử lý gia tăng + partition pruning theo date/hour đã xử lý
Lỗi hết bộ nhớ (OOM) khi chạy Spark cục bộ	Giảm shuffle.partitions, cấu hình bộ nhớ driver/executor hợp lý

CHƯƠNG 7. TẦNG LƯU TRỮ VÀ PHỤC VỤ

7.1 Tổng quan

Tầng lưu trữ và phục vụ gồm ba thành phần: **HDFS** làm kho dữ liệu thô cho tầng lô; **MongoDB Atlas** (dịch vụ cloud bên ngoài cluster) làm kho phục vụ chứa các chỉ số kết quả; và **Dashboard** (FastAPI + React) trực quan hóa dữ liệu cho người dùng.



7.2 Kho dữ liệu thô — HDFS

Dữ liệu tweet thô lưu trên HDFS dưới định dạng **JSON Lines**, truy cập qua **WebHDFS** (thư viện `hdfs InsecureClient`) để không cần binary Hadoop cục bộ. Lớp `HDFSStorageClient` ghi theo **cấu trúc phân vùng coin/ngày/giờ**:

```
def store_raw_tweets(self, coin_symbol, tweets, event_time=None):
    ts = event_time or datetime.utcnow()
    coin = coin_symbol.upper().replace("$", "")
    partition_dir = (f"{self.config.base_dir}/raw_tweets"
f"/coin={coin}/date={ts:%Y-%m-%d}/hour={ts:%H}")
    self.ensure_dir(partition_dir)
    file_path = f"{partition_dir}/{ts:%Y%m%dT%H%M%S}.jsonl"
    self.write_json_lines(tweets, file_path, overwrite=True)
    return file_path
```

Mã 7.1. Ghi tweet xuống HDFS theo phân vùng (`hdfs_client.py`)

Phân vùng theo coin/ngày/giờ cho phép tầng lô đọc chọn lọc (partition pruning), tăng hiệu năng và giảm I/O. Cấu hình HDFS (`WEBHDFS_URL`, `HDFS_USER`, `HDFS_BASE_DIR`) lấy từ biến môi trường.

7.3 Kho phục vụ — MongoDB

Hệ thống dùng **MongoDB Atlas** — dịch vụ MongoDB được quản lý trên cloud (tương tự Kafka Aiven), kết nối qua URI `mongodb+srv://...` lưu trong Secret — không triển khai MongoDB StatefulSet trong cluster. Atlas lưu các collection kết quả do tầng dòng và tầng lô ghi vào. Việc

dùng cơ sở dữ liệu **NoSQL hướng tài liệu** phù hợp với dữ liệu chỉ số có cấu trúc linh hoạt và cần truy vấn nhanh cho dashboard.

Collection	Nguồn ghi	Nội dung
speed_trend_metrics	Speed Layer	Chỉ số xu hướng thời gian thực + cò spike
speed_bad_records	Speed Layer	Thống kê bản ghi lỗi theo cửa sổ
batch_sentiment_metrics	Batch Layer	Cảm xúc/Fear&Greed theo coin (kèm whale/retail)
batch_trend_spikes	Batch Layer	Đột biến thảo luận theo lô (z-score)
alerts	Batch Layer	Cảnh báo spam/bot
batch_job_runs	Batch Layer	Nhật ký kiểm toán mỗi lần chạy job

7.4 Dashboard Backend (FastAPI)

Backend là ứng dụng **FastAPI** chạy bằng uvicorn (công 8000), kết nối MongoDB qua pymongo theo mô hình singleton (khởi tạo một lần trong vòng đời ứng dụng - lifespan). Backend cung cấp các REST API tổng hợp dữ liệu bằng **aggregation pipeline** của MongoDB. Ví dụ truy vấn top coin theo trend_score trong 1 giờ:

```
top = db.speed_trend_metrics.aggregate([
    {"$match": {"window_start": {"$gte": cutoff(1)}}},
    {"$group": {"_id": "$symbol",
        "max_trend": {"$max": "$trend_score"},
        "sum_mention": {"$sum": "$mention_count"}}},
    {"$sort": {"max_trend": -1}}, {"$limit": 1}])
```

Mã 7.2. Truy vấn tổng hợp KPI (backend/main.py)

Endpoint	Chức năng
/health	Kiểm tra kết nối Mongo và đếm document từng collection
/api/summary	Thẻ KPI: top coin, mention 1h, Fear&Greed, số spike/alert
/api/trends/batch	Bảng xếp hạng coin theo cảm xúc (tăng lô)
/api/trends/speed	Top trending theo thời gian thực (tăng dòng)
/api/sentiment/{coin}	Lịch sử cảm xúc của một coin

/api/spikes/batch, /api/spikes/speed	Danh sách đột biến (lô và dòng)
/api/alerts, /api/quality/bad-records, /api/jobs/history	Cảnh báo, dữ liệu lỗi, lịch sử job

Một phản hồi thực tế của /api/summary:

```
{ "top_trending_coin": "BTC", "top_trend_score": 23.0,
  "total_mentions_1h": 225, "avg_fear_greed": 56.9,
  "active_alerts": 961, "active_spikes": 0 }
```

Mã 7.3. Ví dụ phản hồi API /api/summary

7.5 Dashboard Frontend (React + Nginx)

Frontend là ứng dụng **React (Vite)** với các trang: **Trending** (KPI, biểu đồ cảm xúc, bản đồ nhiệt, bảng xếp hạng coin), **Alerts** và **Pipeline** (trạng thái batch/speed, lịch sử job), dùng thư viện biểu đồ recharts. Dữ liệu lấy qua custom hook **tự làm mới định kỳ (polling)** với chu kỳ khác nhau:

```
export const useSummary = () => useApiData("/api/summary", 15_000);
export const useSpeedTrends = (h=1, l=20) =>
  useApiData(`/api/trends/speed?hours=${h}&limit=${l}`, 10_000);
export const useBatchTrends = (h=24, l=20) =>
  useApiData(`/api/trends/batch?hours=${h}&limit=${l}`, 60_000);
```

Mã 7.4. Hook tự làm mới dữ liệu (useDashboard.js)

Khi đóng gói, frontend được build thành tệp tĩnh và phục vụ bằng **Nginx**; Nginx đồng thời **reverse-proxy** /api sang backend để frontend và API cùng nguồn (same-origin), tránh lỗi CORS:

```
location /api/ {
    proxy_pass http://dashboard-backend:8000; # tên Service nội bộ K8s
}
location / {
    try_files $uri $uri/ /index.html; # SPA fallback
}
```

Mã 7.5. Cấu hình reverse-proxy Nginx (nginx.conf)

7.6 Khó khăn gặp phải và cách khắc phục

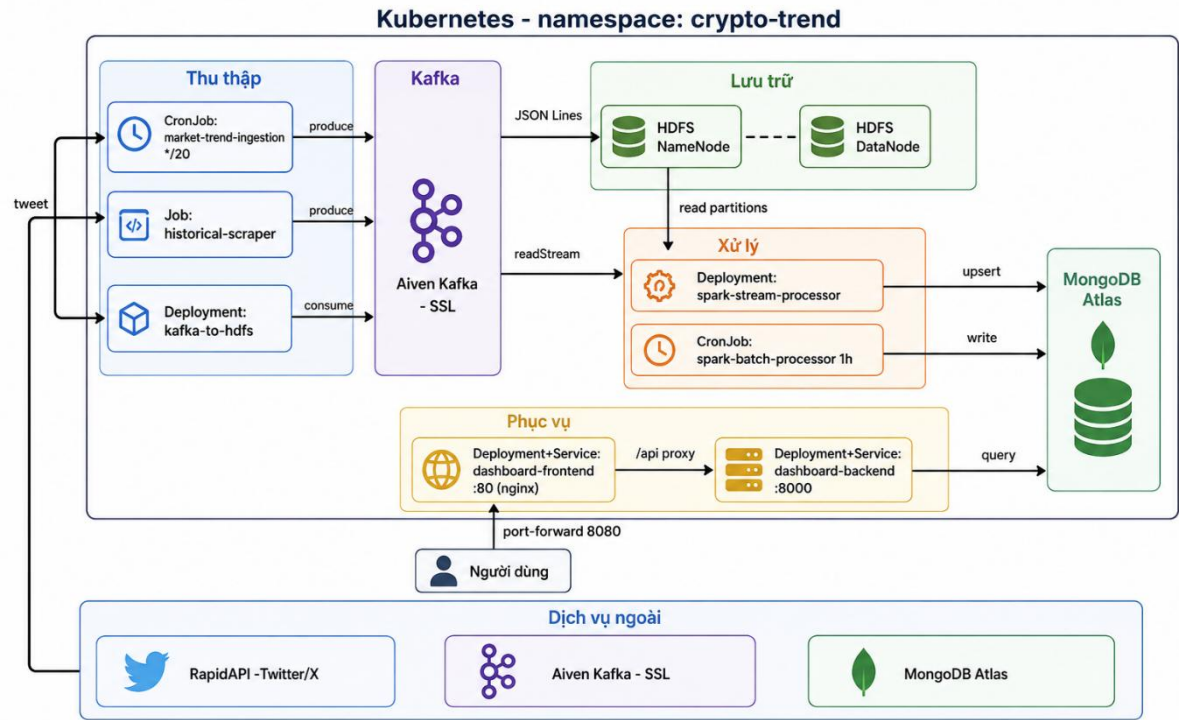
Khó khăn	Cách khắc phục
Lỗi CORS khi frontend gọi API khác nguồn	Dùng Nginx phục vụ tĩnh + reverse-proxy /api → cùng nguồn (same-origin)

API trả 404 khi coin chưa có dữ liệu bị hiển thị như lỗi nghiêm trọng	Coi 404 là 'chưa có dữ liệu', hiển thị trạng thái trung tính thay vì báo đỏ
Service ClusterIP không truy cập được từ máy người dùng	Dùng kubectl port-forward ánh xạ cổng cục bộ tới Service

CHƯƠNG 8. TRIỂN KHAI HỆ THỐNG TRÊN KUBERNETES

8.1 Tổng quan và lý do lựa chọn

Toàn bộ hệ thống được triển khai trên **Kubernetes (K8s)**. So với Docker Compose, Kubernetes mang lại tự phục hồi (self-healing), quản lý tác vụ định kỳ qua CronJob, quản lý cấu hình bí mật qua Secret, khả năng mở rộng và phân tách tài nguyên theo namespace — phù hợp cho hệ thống nhiều thành phần. Cụm dùng **Kind (Kubernetes in Docker)** gồm ba node (một control-plane, hai worker, runtime containerd); ứng dụng nằm trong namespace **crypto-trend**.



8.2 Các đối tượng Kubernetes sử dụng

Đối tượng	Đặc điểm	Áp dụng
Namespace	Phân vùng tài nguyên	crypto-trend
Secret	Lưu thông tin nhạy cảm	Khóa API, chứng chỉ Kafka, URI Mongo
Deployment	Tiến trình liên tục, tự phục hồi	kafka-to-hdfs, stream, dashboard
StatefulSet	Ứng dụng có trạng thái + ổ đĩa bền	HDFS namenode/datanode (MongoDB dùng Atlas cloud, không deploy StatefulSet)
CronJob	Tác vụ theo lịch	Thu thập market, batch Spark
Job	Tác vụ chạy một lần	Cào dữ liệu lịch sử
Service	Điểm truy cập mạng ổn định	dashboard-backend/frontend

8.3 Đóng gói Docker

crypto-ingestion (python:3.11-slim) — dùng chung cho Job, CronJob và kafka-to-hdfs.

crypto-stream / crypto-batch — python:3.11-slim kèm Java Runtime để chạy Apache Spark.

crypto-dashboard-backend — image FastAPI (uvicorn, cổng 8000).

crypto-dashboard-frontend — image **multi-stage**: build React/Vite bằng node:20-alpine, phục vụ bằng nginx:1.27-alpine kèm reverse-proxy /api.

Tất cả đặt imagePullPolicy: Never vì image được nạp sẵn vào cụm.

8.4 Quy trình build và nạp image

Do Kind chạy trên Docker Desktop và công cụ kind không nhận diện được cụm trong context docker hiện hành (không dùng được 'kind load'), hệ thống nạp image bằng cách **lưu ra tệp và nhập vào containerd của từng node worker**:

```
docker build -t crypto-stream:v1 -f Dockerfile.stream .
docker save crypto-stream:v1 -o stream.tar
docker cp stream.tar desktop-worker:/stream.tar
docker exec desktop-worker ctr -n k8s.io images import /stream.tar
# lặp lại cho node desktop-worker2
```

Mã 8.1. Build và nạp image vào containerd của node

8.5 Quản lý cấu hình bí mật (Secret)

Hai Secret được tạo bằng kubectl create secret (không commit vào mã nguồn): **ingestion-env-secret** (khóa API, host Kafka, topic, MONGO_URI, WEBHDFS_URL...) nạp qua envFrom; **kafka-ssl-secret** (ba chứng chỉ) mount vào /etc/kafka/certs. Cách dùng trong manifest:

```
envFrom:
  - secretRef:
      name: ingestion-env-secret
volumeMounts:
  - name: kafka-ssl-certs
    mountPath: /etc/kafka/certs
    readOnly: true
```

Mã 8.2. Nạp Secret vào container

8.6 Một số manifest tiêu biểu

CronJob thu thập thị trường (chạy mỗi 20 phút):

```
kind: CronJob
metadata: { name: market-trend-ingestion, namespace: crypto-trend }
spec:
  schedule: "*/20 * * * *"
  timeZone: "Asia/Ho_Chi_Minh"
  concurrencyPolicy: Forbid
  jobTemplate: { ... command: ["python", "src/ingestion/twitter_client.py"] }
```

Mã 8.3. CronJob thu thập (ingestion.yaml)

Deployment tầng dòng (ghi thẳng MongoDB):

```
command: ["python", "src/processing/speed_layer/stream_job.py",
  "--output-sink", "mongo"]
```

Mã 8.4. Lệnh chạy stream với sink MongoDB (processing.yaml)

8.7 Lập lịch và truy cập

Tác vụ	Lịch (cron)	Tần suất
market-trend-ingestion	* / 20 * * * *	Mỗi 20 phút
spark-batch-processor	0 * * * *	Mỗi 1 giờ
spark-stream-processor	(Deployment)	Liên tục, thời gian thực

CronJob đặt timeZone 'Asia/Ho_Chi_Minh' và concurrencyPolicy: Forbid để tránh chồng chéo. Do Service là ClusterIP, truy cập Dashboard qua **kubectl port-forward** (localhost:8080 → dashboard-frontend:80).

CHƯƠNG 9. KẾT QUẢ THỰC HIỆN VÀ ĐÁNH GIÁ

9.1 Kết quả triển khai

Hệ thống đã triển khai và vận hành thành công trên cụm Kubernetes với đầy đủ bốn nhóm thành phần (thu thập, xử lý dòng, xử lý lô, phục vụ). Toàn bộ Pod ở trạng thái Running; CronJob kích hoạt đúng lịch và Job hoàn tất (Completed). Trạng thái thực tế của cụm:

1	NAME	READY	STATUS	RESTARTS	AGE	
2	dashboard-backend-64b8c946cc-4kq8c	1/1	Running	10 (3h40m ago)	2d5h	
3	dashboard-frontend-6dbb78977-hbwqf	1/1	Running	0	91m	
4	datanode-0	1/1	Running	7 (7h43m ago)	5d15h	
5	historical-scraper-2rb8w	0/1	Completed	0	5d7h	
6	kafka-to-hdfs-85d4fc4b7f-vl2hm	1/1	Running	6 (7h43m ago)	2d6h	
7	market-trend-ingestion-29675120-pjhk6	0/1	Completed	0	45m	
8	market-trend-ingestion-29675140-g88n9	0/1	Completed	0	25m	
9	market-trend-ingestion-29675160-bm7lk	0/1	Completed	0	5m4s	
10	namenode-0	1/1	Running	7 (7h43m ago)	5d15h	
11	spark-batch-processor-29675040-bgtjn	0/1	Completed	0	66m	
12	spark-batch-processor-29675100-hqrzf	0/1	Completed	0	65m	
13	spark-batch-processor-29675160-7k5qb	0/1	Completed	0	5m4s	
14	spark-stream-processor-8c6f4475-6lq6q	1/1	Running	0	155m	
15	---					
16	CronJob & Service ---					
17	NAME	SCHEDULE	TIMEZONE	SUSPEND	ACTIVE	LAST SCHED
18	cronjob.batch/market-trend-ingestion	* / 20 * * * *	Asia / Ho_Chi_Minh	False	0	5m5s
19	cronjob.batch/spark-batch-processor	0 * * * *	Asia / Ho_Chi_Minh	False	0	5m5s
20						
21	NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
22	service/dashboard-backend	ClusterIP	10.96.127.185	<none>	8000 / TCP	2d5h
23	service/dashboard-frontend	ClusterIP	10.96.101.115	<none>	80 / TCP	2d5h
24	service/datanode	ClusterIP	10.96.124.150	<none>	9864 / TCP	5d15h
25	service/namenode	ClusterIP	10.96.65.19	<none>	9000 / TCP, 9870 / TCP	5d15h

Thành phần	Loại	Trạng thái
spark-stream-processor	Deployment	Running (ghi MongoDB liên tục)
kafka-to-hdfs	Deployment	Running
dashboard-backend / frontend	Deployment	Running
namenode / datanode	StatefulSet	Running
market-trend-ingestion	CronJob * / 20	Kích hoạt đúng lịch (Completed mỗi lần)
spark-batch-processor	CronJob 0 * * * *	Kích hoạt đúng lịch (Completed mỗi lần)

9.2 Số liệu thực nghiệm

Số liệu đo trực tiếp trên hệ thống đang vận hành (sau nhiều ngày chạy):

Chỉ số	Giá trị đo được
Bản ghi xu hướng thời gian thực (speed_trend_metrics)	≈ 3.392 document
Bản ghi cảm xúc theo lô (batch_sentiment_metrics)	≈ 3.172 document
Cảnh báo spam đã sinh (alerts)	≈ 961 document
Số coin được phân tích (batch)	8 coin: BTC, ETH, SOL, XRP, ADA, BNB, DOGE, AVAX
Thời lượng một lần chạy batch (38 nhóm coin)	≈ 15 giây
Mentions/1h hiển thị trên Dashboard (thời điểm đo)	225 lượt
Số lần chạy batch đã ghi nhật ký (batch_job_runs)	19 lần, đều status=success

9.3 Đánh giá hệ thống

Tiêu chí	Kết quả	Đánh giá
Tính end-to-end	Dữ liệu chảy thông suốt qua 4 tầng	Đạt
Độ tin cậy thu thập	Xoay vòng khóa API, commit offset thủ công	Không mất dữ liệu
Khả năng tự phục hồi	Pod tự khởi động lại sau sự cố	Đạt (K8s + checkpoint)
Nhất quán dữ liệu dòng	Upsert idempotent theo cửa sổ	Hiệu quả exactly-once
Hiệu năng batch	Xử lý gia tăng + partition pruning, ~15s/lần	Tốt
Trực quan hóa	Dashboard cập nhật gần thời gian thực	Đạt

9.4 Kỹ thuật Spark đã vận dụng

Hệ thống thể hiện nhiều kỹ thuật Spark mức trung cấp: biến đổi nhiều giai đoạn và chuỗi thao tác phức tạp; **UDF tùy biến** cho lọc spam và chấm điểm cảm xúc; **cửa sổ thời gian** (sliding/tumbling) kèm **watermark** và xử lý dữ liệu đến trễ; **quản lý trạng thái và checkpoint**; tổng hợp có điều kiện; **partition pruning** và **xử lý gia tăng** ở tầng lô; cùng các thống kê (z-score, độ lệch chuẩn, growth-rate) phục vụ phát hiện spike.

9.5 Khó khăn lớn gặp phải và cách khắc phục

Khó khăn	Nguyên nhân	Cách khắc phục
Tăng dòng không ghi được MongoDB	Stream chạy ở chế độ console (mặc định), thiếu tham số sink	Bổ sung --output-sink mongo vào lệnh chạy Deployment
Không nạp được image vào cụm	kind không nhận diện cụm trong context docker hiện hành	Lưu image ra tar và import vào containerd của từng node bằng ctr
Pod báo ErrImageNeverPull	Image chưa được nạp lên node mà pod được lập lịch	Import image vào CẢ hai node worker (pod chạy trên worker)
port-forward bị đứt khi pod restart	port-forward bám vào một pod cụ thể	Khởi động lại port-forward sau khi pod mới Ready

CHƯƠNG 10. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

10.1 Kết luận

Project đã xây dựng thành công một **hệ thống phân tích xu hướng tiền mã hóa hoàn chỉnh theo kiến trúc Lambda**: từ thu thập dữ liệu mạng xã hội, xử lý song song theo dòng và theo lô bằng Apache Spark, lưu trữ trên HDFS/MongoDB, đến trực quan hóa bằng Dashboard web. Toàn bộ hệ thống được container hóa và triển khai trên Kubernetes, chứng minh khả năng vận hành một pipeline dữ liệu lớn nhiều thành phần với tính sẵn sàng và tự phục hồi. Các kết quả chính: làm chủ luồng dữ liệu end-to-end; vận dụng đa dạng kỹ thuật Spark; thiết kế cơ chế chịu lỗi; và triển khai thực tế trên Kubernetes.

10.2 Bài học kinh nghiệm

- **Tách rời qua hàng đợi là chìa khóa**: việc dùng Kafka giúp các tầng phát triển, triển khai và khắc phục sự cố độc lập, hạn chế lỗi lan truyền giữa các thành phần.
- **Cấu hình mặc định có thể gây lỗi âm thầm**: sự cố tăng dòng ghi ra console thay vì MongoDB cho thấy cần kiểm chứng từng tham số vận hành (output sink) chứ không chỉ xem pod 'Running'.
- **Tính bất biến (idempotency) quan trọng hơn 'exactly-once' tuyệt đối**: ghi bằng upsert theo khóa cửa sổ giúp đạt kết quả đúng kể cả khi dữ liệu bị xử lý lại — đơn giản và bền vững hơn việc cố đạt exactly-once thuần túy.
- **Hiểu rõ môi trường triển khai**: việc 'kind load' không hoạt động buộc phải hiểu cơ chế containerd/ctr bên dưới — kiến thức nền giúp xử lý sự cố nhanh hơn là phụ thuộc công cụ.
- **Quan sát được (observability) là cần thiết**: log, healthcheck và collection kiểm toán (batch_job_runs, speed_bad_records) giúp phát hiện và chẩn đoán sự cố kịp thời.

10.3 Hạn chế

- Phân tích cảm xúc dựa trên từ điển, độ chính xác chưa cao bằng mô hình học sâu và chưa tối ưu cho tiếng lóng/biểu tượng của cộng đồng crypto.
- Cụm Kubernetes triển khai cục bộ (Kind) phục vụ thử nghiệm, chưa lên hạ tầng đám mây thật.
- Phụ thuộc gói RapidAPI miễn phí với giới hạn truy vấn, ảnh hưởng tần suất và độ phủ thu thập.
- Chưa khai thác các kỹ thuật join nâng cao (broadcast/sort-merge), MLlib và GraphFrames.

10.4 Hướng phát triển

- **Nâng cấp phân tích**: thay phân tích cảm xúc từ điển bằng mô hình học máy/học sâu (hoặc Spark MLlib); bổ sung phân tích mạng lưới tương tác bằng GraphFrames.
- **Triển khai đám mây**: đưa cụm lên VPS/Cloud (EKS/GKE) để vận hành ổn định và truy cập từ xa.
- **Dự báo**: dùng dữ liệu lịch sử để dự báo xu hướng/biến động sớm; tích hợp tín hiệu giá.
- **Mở rộng nguồn**: thu thập thêm từ Reddit, Telegram và nâng cấp gói API để tăng độ phủ.